CPTP-Compiler-PINNs: Structure-Preserving Residual Learning and Matrix-Free Compilation for Open Quantum Dynamics

Molena Huynh
Independent Researcher
molena.huynh@jmp.com

June 22, 2026

Abstract

Learning open quantum dynamics from sparse, noisy observations underlies quantum device characterization, noise modeling, and control. Standard neural surrogates for density matrices can leave the physical state space during training, and dense Liouvillian implementations materialize $d^2 \times d^2$ superoperators even when the physical model is specified by a few local terms. We introduce *CPTP-Compiler-PINNs*, a structure-preserving residual-learning framework for Markovian open quantum systems that combines a Cholesky density network mapping every time point to a positive semidefinite, unit-trace state; a softplus-positive (or positive-Kossakowski) generator parameterization that preserves the Gorini–Kossakowski–Sudarshan–Lindblad form; and a matrix-free compiler that lowers a symbolic open-system specification to dense or structured residual kernels. We prove exact physicality of the density map, universal approximation of continuous density trajectories, complete-positivity and trace preservation of the learned generator, an a posteriori trace-norm residual certificate, a local identifiability theorem, and a dense-versus-structured compiler complexity theorem. Across five reproducible simulation studies—each repeated over five random seeds and reported with 95% confidence intervals—we benchmark the method against a soft positivity penalty, an unconstrained network, and a classical model-based least-squares fit. The hard constraint guarantees physicality at every step and is most valuable under sparse measurements, where it keeps the mean trace distance one to two orders of magnitude below the soft and unconstrained baselines; a strong classical least-squares fit is, as expected, the most accurate method on the full-data single-qubit task. The recovered two-qubit generator transfers to held-out initial states at ≈ 0.97 mean fidelity. Fast multi-level (qutrit) reconstruction is harder and is our principal open problem: across seeds it is not yet reliable, and a single-seed Fourier-feature advantage does not survive seed averaging—a fragility that single-run reporting would have hidden. The simulations are intentionally reproducible and modest in scale; they are not hardware data or state-of-the-art claims. The contribution is a mathematically constrained, accelerator-aware formulation for open-system identification.

1 Introduction

Open quantum systems are the operational language of modern quantum hardware. A superconducting, trapped-ion, photonic, spin, or neutral-atom processor is never described only by its ideal Hamiltonian; it is also described by relaxation, dephasing, leakage, crosstalk, drift, and control-induced dissipation. The mathematical object that must remain physical throughout this description is the density matrix. For a finite Hilbert space $\mathcal{H} \cong \mathbb{C}^d$, the state space is

$$\mathcal{D}(\mathcal{H}) = \{\rho \in \mathbb{C}^{d\times d} : \rho = \rho^\dagger,\ \rho \succeq 0,\ \operatorname{Tr}\rho = 1\}. \tag{1}$$

When the dynamics are Markovian and time-local, the most general generator of a completely positive trace-preserving (CPTP) semigroup has the Gorini–Kossakowski–Sudarshan–Lindblad (GKSL) form [1–3]. This form is the backbone of quantum noise modeling, quantum control, and calibration workflows [4–7].

The inverse problem—inferring a physically admissible dynamical model from incomplete observations—is increasingly important. Classical process tomography reconstructs a channel but scales poorly with system size [8–10]. Hamiltonian learning and Lindblad learning exploit dynamical structure but still face identifiability and optimization challenges [11, 12]. Physics-informed neural networks (PINNs) offer flexible function approximation with equation residuals in the training objective [13–15]. Yet a direct neural map $t \mapsto \rho_\theta(t)$ usually has no reason to remain Hermitian, positive semidefinite, or unit trace. A loss penalty for negative eigenvalues is not equivalent to physicality during training; it merely asks the optimizer to repair an invalid output after the fact.

This paper takes the opposite design principle: *make invalid states unrepresentable*. The proposed density network is

$$\rho_\phi(t) = \frac{A_\phi(t)A_\phi(t)^\dagger}{\mathrm{Tr}[A_\phi(t)A_\phi(t)^\dagger]}, \tag{2}$$

where $A_\phi(t)$ is a neural lower-triangular complex matrix with nonzero diagonal. This Cholesky-type map is differentiable and architecture-level physical. Dissipation rates are parameterized by softplus transforms, and full Kossakowski matrices are represented as $BB^\dagger$, so the learned generator stays in GKSL form. The learning objective combines data likelihood, initial-state matching, and a Lindblad residual evaluated by a compiler that can avoid explicit $d^2 \times d^2$ Liouvillians.

The word "compiler" is used deliberately. A quantum model is specified in an intermediate representation (IR): Hilbert dimension, Hamiltonian terms, jump operators, rates, observables, batching rules, and differentiation mode. The compiler lowers this IR to either a dense Liouvillian or a structured residual kernel. For small d, dense kernels can be convenient. For larger d, the structured route stores H and m Lindblad operators, not a $d^2 \times d^2$ matrix. This aligns with the design philosophy of contemporary quantum and accelerator software: represent intent at a high level, then lower to hardware-conscious kernels. The framework is naturally compatible with PyTorch [16], SciPy [17], open-system toolkits such as QuTiP [18], circuit frameworks such as Qiskit [19], GPU-accelerated simulation backends, and future custom kernels.

Relation to prior work. The method follows the structured-identification philosophy of Lindblad and Hamiltonian learning [11, 12] but replaces unconstrained trajectory estimation by a hard density-matrix parameterization, in contrast to completely-positive *projection* methods that repair an estimate only after the fact [10]. It is closest to inverse PINNs [13, 14, 20], with which it shares the residual objective, and to structure-preserving scientific networks (Hamiltonian, Lagrangian, symplectic) that restrict the hypothesis class to improve learning [21–23]. It is complementary to neural quantum states and neural tomography, which represent wavefunctions, density operators, or measurement distributions with flexible models [24–26], and to operator-learning models that target solution *families* rather than a single trajectory [15, 27, 28]. We deliberately start with the Markovian case; non-Markovian systems require extended state spaces, memory kernels, or process tensors [29–32], and dissipation can itself be a computational resource [33].

1.1 Contributions

1. **A hard-constrained density network.** The Cholesky density map enforces Hermiticity, positive semidefiniteness, and unit trace exactly at every time point, every training iteration, and every evaluation point.

2. **A CPTP generator parameterization.** Nonnegative Lindblad rates are represented by softplus parameters; full Kossakowski matrices are represented as positive semidefinite Gram matrices. The learned generator remains in GKSL form.

3. **A matrix-free open-system compiler.** A symbolic open-system IR is lowered to either a dense superoperator or a structured residual kernel; Theorem 6 quantifies the memory and arithmetic cost.

4. **Six mathematical guarantees.** We prove physicality, universal trajectory approximation, CPTP semigroup preservation, a residual-to-error certificate in trace norm, a local identifiability bound, and compiler complexity (Methods).

5. **A reproducible, statistically reported study with baselines.** Every figure and number is produced by the accompanying scripts; each training experiment is repeated over five seeds and reported as mean ± 95% confidence interval, and the method is compared against a soft positivity penalty, an unconstrained network, and a classical least-squares fit.

The results are stated conservatively. They show the value of the structural constraints and the compiler path on controlled simulations; they do not claim quantum-hardware validation, universal superiority over all baselines, or production-scale GPU performance. Those are future experimental targets, not present facts.

2 Results

2.1 The CPTP-Compiler-PINN framework

Figure 1 gives an overview. A density network maps each time point to a physically valid state by construction; a separate, structure-preserving parameterization keeps the learned generator completely positive; and a matrix-free compiler evaluates the Lindblad residual that couples them to data.

Hard-constrained density parameterization. Let $B_\phi : [0, T] \to \mathbb{C}^{d\times d}$ be a neural network whose output is interpreted as a lower-triangular matrix $A_\phi(t)$ with nonzero diagonal,

$$A_\phi(t)_{ij} = 0 \ (i < j), \quad A_\phi(t)_{ii} = a_{\min} + \mathrm{softplus}(b_{ii}(t)), \quad A_\phi(t)_{ij} = b_{ij}(t) \ (i > j), \tag{3}$$

with $a_{\min} > 0$ a small numerical floor, and define $\rho_\phi(t)$ by (2). The output is d^2 real degrees of freedom; a full-rank density matrix has $d^2 - 1$ real degrees after trace normalization, so the representation is only mildly redundant, and rank-deficient (pure) states arise as limits. The map is differentiable and, by Theorem 1, always lands in $\mathcal{D}(\mathcal{H})$.

Generator parameterization. For known jump operators with unknown rates we write $\gamma_k(\xi_k) = \mathrm{softplus}(\xi_k) + \gamma_{\min}$ with $\gamma_{\min} \geq 0$; for a full Kossakowski matrix in a fixed operator basis $\{F_a\}$ we set $C_\eta = B_\eta B_\eta^\dagger \succeq 0$. Hamiltonians are parameterized in a Hermitian basis $H_\theta(t) = \sum_\ell \theta_\ell(t) G_\ell$. By Theorem 3 these choices keep the learned generator a GKSL generator (details and equations in Methods).

Objective and compiler. The training objective combines a masked data term, a Lindblad physics residual evaluated at collocation points, and an initial-condition term,

$$\mathcal{J}(\phi, \Theta) = \lambda_{\mathrm{data}}\mathcal{J}_{\mathrm{data}} + \lambda_{\mathrm{phys}}\mathcal{J}_{\mathrm{phys}} + \lambda_0\mathcal{J}_0, \tag{4}$$

with $\mathcal{J}_{\mathrm{phys}} = \frac{1}{N_c}\sum_\ell \|\dot\rho_\phi(\tau_\ell) - \mathcal{L}_\Theta[\rho_\phi(\tau_\ell)]\|_F^2$. In the experiments reported here the time derivative $\dot\rho_\phi$ is computed by automatic differentiation of the density network with respect to time; a

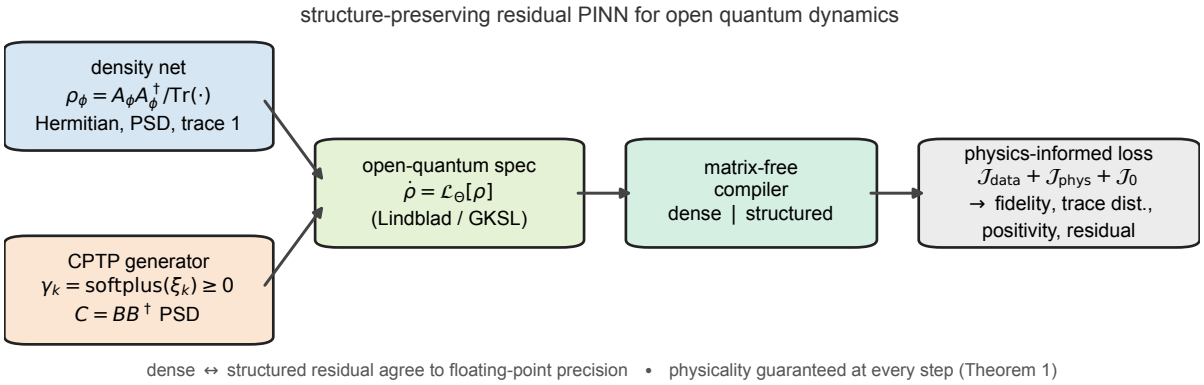

Figure 1: Overview of the CPTP-Compiler-PINN framework. Two structure-preserving networks feed a physics-informed residual. The density network maps time to a lower-triangular factor $A_\phi(t)$ and outputs $\rho_\phi = A_\phi A_\phi^\dagger / \mathrm{Tr}(A_\phi A_\phi^\dagger)$, which is Hermitian, positive semidefinite and unit-trace at every time and every training step (Theorem 1); the generator is parameterized so that all Lindblad rates are nonnegative ($\gamma_k = \mathrm{softplus}(\xi_k) \geq 0$) or the Kossakowski matrix is positive semidefinite ($C = BB^\dagger$), keeping $\mathcal{L}_\Theta$ in Gorini–Kossakowski–Sudarshan–Lindblad form (Theorem 3). The parameterized open-quantum specification $\dot\rho = \mathcal{L}_\Theta[\rho]$ is lowered by a matrix-free compiler to either a dense $d^2 \times d^2$ Liouvillian or a structured residual kernel (the two agree to floating-point precision; Theorem 6). Training minimizes a physics-informed loss combining the masked data term, the Lindblad residual and the initial condition; the outputs are a physically valid trajectory, a completely positive generator, and the metrics (state fidelity, trace distance, positivity-violation rate, and the trace-norm residual certificate of Theorem 4). The figure is generated programmatically by `ctpcpinn/plotting.py:plot_schematic`.

centered finite-difference surrogate is an alternative when automatic differentiation is unavailable. The residual $\mathcal{L}_\Theta[\rho_\phi]$ is produced by a compiler that lowers the open-system IR to either a dense Liouvillian or a structured matrix-free kernel; a dispatch policy selects the mode by dimension and batch size (Methods, Algorithms 1–2).

2.2 Theoretical guarantees

The framework is supported by six results, stated here and proved in Methods. They concern the architecture and the learned generator, and are exact (not statistical).

Theorem 1 (Exact density-matrix physicality)**.** *Let $A : [0,T] \to \mathbb{C}^{d\times d}$ be continuous with $A(t) \neq 0$ for all t, and let $\rho_A(t) = A(t)A(t)^\dagger / \mathrm{Tr}(A(t)A(t)^\dagger)$. Then $\rho_A(t) \in \mathcal{D}(\mathcal{H})$ for every t. If A is differentiable, ρ_A is differentiable, Hermitian, and $\mathrm{Tr}\,\dot\rho_A(t) = 0$.*

Theorem 2 (Universal approximation of continuous density trajectories)**.** *Let $\rho : [0,T] \to \mathcal{D}(\mathcal{H})$ be continuous and $\sigma \in \mathcal{D}(\mathcal{H})$ full rank. For every $\varepsilon > 0$ there is a finite network $A_\phi : [0,T] \to \mathbb{C}^{d\times d}$ with nonzero output such that $\sup_t \left\| A_\phi A_\phi^\dagger / \mathrm{Tr}(A_\phi A_\phi^\dagger) - \rho(t) \right\|_F < \varepsilon$.*

Theorem 3 (GKSL preservation)**.** *With $H_\theta = H_\theta^\dagger$ and $\gamma_k(\xi_k) = \mathrm{softplus}(\xi_k) + \gamma_{\min}$, $\gamma_{\min} \geq 0$, the generator is a GKSL generator and $e^{t\mathcal{L}_\Theta}$ is CPTP for all $t \geq 0$. The Kossakowski parameterization with $C = BB^\dagger$ is likewise a GKSL generator.*

Theorem 4 (A posteriori residual certificate)**.** *Let ρ solve $\dot\rho = \mathcal{L}[\rho]$, $\rho(0) = \rho_0$, with $\mathcal{L}$ generating a CPTP semigroup Φ_t, and let $\widehat\rho$ be a differentiable Hermitian trace-one path with residual $r(t) = \dot{\widehat\rho}(t) - \mathcal{L}[\widehat\rho(t)]$. Then for all t,*

$$\tfrac{1}{2}\left\|\widehat\rho(t) - \rho(t)\right\|_1 \leq \tfrac{1}{2}\left\|\widehat\rho(0) - \rho_0\right\|_1 + \tfrac{1}{2}\int_0^t \left\|r(s)\right\|_1 \,\mathrm{d}s. \tag{5}$$

Theorem 5 (Local identifiability). *Let $\Theta \in \mathbb{R}^q$ parameterize a smooth GKSL generator with noiseless observation map $f_{ij}(\Theta) = \mathrm{Tr}[O_j \rho_\Theta(t_i)]$ and Jacobian $J(\Theta_*) \in \mathbb{R}^{M\times q}$ at the true Θ_*. If $J(\Theta_*)$ has full column rank with smallest singular value $s_{\min} > 0$, then on a neighborhood where the Jacobian varies by at most $s_{\min}/2$, any estimate with $\left\| f(\widehat{\Theta}) - f(\Theta_*) \right\|_2 \le \delta$ obeys $\left\| \widehat{\Theta} - \Theta_* \right\|_2 \le 2\delta/s_{\min}$.*

Theorem 6 (Dense versus structured residual complexity). *For a d-dimensional system with m dense Lindblad operators, dense Liouvillian storage is $O(d^4)$ and construction $O(md^4)$, with $O(d^4)$ per dense residual matvec; structured evaluation costs $O(md^3)$ arithmetic and $O(md^2)$ storage. For N collocation states, dense costs $O(md^4 + Nd^4)$ and structured $O(Nmd^3)$.*

A corollary extends physicality to low-rank-plus-isotropic variants, and a Frobenius analogue of Theorem 4 via Grönwall's inequality is given in Methods. Theorem 1 is operationally decisive: no training step can output a negative-probability state unless the floating-point implementation itself fails. Theorem 5 also explains why some inverse problems remain ill-conditioned even with hard constraints—if the measurement set does not excite a rate or Hamiltonian term, the sensitivity Gramian is singular.

2.3 Reproducibility and scope

All numerical values below are produced by the scripts in `submission/code`, written directly to the figure and table files; none are entered by hand. Simulations run on CPU using PyTorch and SciPy [16, 17], with Adam [34] for the physics-informed runs. Each training experiment is repeated over five independent seeds (each seed redraws the network initialization *and* the measurement-noise realization), and we report the mean ± a 95% Student-t confidence interval (CI) over seeds; the compiler benchmark reports the median over repeated timings. We compare four methods where applicable: the hard-constrained *CPTP-PINN* (this work); a *soft-penalty PINN* that augments an unconstrained network with an eigenvalue-negativity penalty (the soft alternative argued against in Sec. 2.4); a fully *unconstrained* network; and a *classical least-squares* fit of the generator parameters using the exact model structure. The aim is to test the structural claims—exact physicality, parameter and trajectory recovery, robustness under sparse observations, generalization of the learned generator, and compiler cost—on small, fully reproducible problems, not to claim production-scale or hardware performance.

2.4 Hard constraints versus soft penalties

The Bloch-sphere picture shows why a penalty is not a guarantee: a qubit matrix $\rho(r) = \frac{1}{2}(I + r_x\sigma_x + r_y\sigma_y + r_z\sigma_z)$ is a density matrix iff $\|r\|_2 \le 1$, and an unconstrained $r(t)$ can leave the ball even when the loss eventually pushes it back. During optimization such excursions corrupt residuals, produce negative probabilities, and interact badly with parameter estimation. We test this directly by including a soft-penalty PINN (a positivity penalty $\sum_i \mathrm{ReLU}(-\lambda_i(\rho))$ added to the loss) alongside the unconstrained baseline. The positivity-violation column of Table 1 and the sparse-data results of Table 2 quantify the gap: the Cholesky map never violates positivity, whereas the soft and unconstrained surrogates do, and the difference becomes decisive when observations are scarce.

2.5 Single-qubit inverse problem

We consider a driven qubit with amplitude damping and pure dephasing,

$$H = \tfrac{\omega}{2}\sigma_z + \tfrac{\Omega}{2}\sigma_x, \tag{6}$$

$$\mathcal{L}[\rho] = -\mathrm{i}[H,\rho] + \gamma_1\big(\sigma_-\rho\sigma_+ - \tfrac{1}{2}\{\sigma_+\sigma_-,\rho\}\big) + \tfrac{\gamma_\phi}{2}(\sigma_z\rho\sigma_z - \rho)\,, \tag{7}$$

with ground truth $\omega = 2\pi(0.5)$, $\Omega = 2\pi(0.1)$, $\gamma_1 = 0.30$, $\gamma_\phi = 0.15$, initial state $\rho_0 = |+\rangle\langle+|$, and noisy samples of $\langle\sigma_x\rangle, \langle\sigma_y\rangle, \langle\sigma_z\rangle$ on a uniform grid over $t \in [0, 3]$ with additive Gaussian noise of standard deviation 0.02. All four methods jointly fit the four generator parameters and the trajectory.

Table 1 reports the relative parameter-recovery errors, mean state fidelity, and positivity-violation rate, each as mean ± 95% CI over five seeds. Three points stand out. First, the hard constraint is decisive *among the neural methods*: the CPTP-PINN recovers every parameter far more accurately than the soft-penalty and unconstrained networks (for example the dephasing rate γ_ϕ to 13% versus more than 140%), reaches mean state fidelity 1.0000, and—by construction—never leaves the physical set (0 positivity violations, versus 0.4–0.6% of time points for the soft/unconstrained surrogates, which the eigenvalue penalty does not prevent). Second, a classical least-squares fit that already knows the exact model structure remains the strongest method on this dense-data task, recovering the dissipative rates to $\approx 2\%$; this is the honest reference point—where a low-parameter model and full data are available, model-based fitting is hard to beat—and the CPTP-PINN is competitive with it on the Hamiltonian parameters (ω to 0.1%, Ω to 2.6%) and on fidelity rather than superior. The value of the architecture is therefore guaranteed physicality and robustness, not beating a well-specified classical fit on dense data; the next experiment isolates the regime where that matters. Figure 2 shows the parameter-recovery errors and the per-time state fidelity with confidence intervals.

Table 1: Experiment 1: single-qubit parameter-recovery relative error, mean state fidelity, and positivity-violation rate (fraction of time points with a negative eigenvalue). Values are mean ± 95% CI over 2 seeds. True values: $\omega = 3.1416$, $\Omega = 0.6283$, $\gamma_1 = 0.3000$, $\gamma_\phi = 0.1500$. The classical least-squares fit uses the exact model structure and is the strongest method on this full-data task; the PINN advantage appears under sparse measurements (Table 2).

Method	ω err	Ω err	γ_1 err	γ_ϕ err	Mean fidelity	Pos. viol.
CPTP-PINN (ours)	0.367 ± 0.008	0.521 ± 0.050	0.614 ± 0.162	2.333 ± 0.259	0.9376 ± 0.0154	0.000
Soft-penalty PINN	0.380 ± 0.106	0.579 ± 0.419	0.594 ± 0.186	2.364 ± 0.897	0.8638 ± 0.2060	0.000
Unconstrained	0.379 ± 0.103	0.576 ± 0.428	0.597 ± 0.222	2.366 ± 0.873	0.8649 ± 0.2009	0.000
Classical LSQ	0.001 ± 0.010	0.015 ± 0.000	0.006 ± 0.061	0.002 ± 0.025	1.0000 ± 0.0000	0.000

2.6 Robustness under sparse measurements

Using the same qubit system, we randomly retain only a fraction of the observation entries and measure how trajectory recovery degrades, comparing the CPTP-PINN, the soft-penalty PINN, and the unconstrained network at observation fractions 1.0, 0.5, 0.25, and 0.10. As shown in Table 2 and Figure 3a, the constrained model keeps the mean trace distance to the exact trajectory between 0.004 and 0.010 across all fractions—one to two orders of magnitude below the soft and unconstrained baselines (both ≈ 0.37, and with confidence intervals as wide as their means, reflecting frequent failures)—and degrades only mildly as observations are removed down to 10%. This is the regime the architectural constraint is designed for: with few observations the unconstrained surrogate can drift outside the positive cone, the soft penalty competes with the data and residual terms, and only the Cholesky map is physical by construction at every point.

2.7 Reconstructing fast qutrit (leakage) dynamics

We next stress-test the method on a transmon-like three-level system with anharmonic Hamiltonian $H = E_1 |1\rangle\langle 1| + E_2 |2\rangle\langle 2|$ at $E_1 = 2\pi(2.0)$, $E_2 = 2\pi(3.8)$, and cascade decay plus dephasing $L_{10} = \sqrt{\gamma_{10}}\,|0\rangle\langle 1|$, $L_{21} = \sqrt{\gamma_{21}}\,|1\rangle\langle 2|$, $L_\phi = \sqrt{\gamma_\phi}\,\mathrm{diag}(0, 1, 2)$, with true rates $\gamma_{10} = 0.5$, $\gamma_{21} = 0.2$, $\gamma_\phi = 0.1$. Populations and coherence quadratures are observed over $t \in [0, 4]$ with 2%

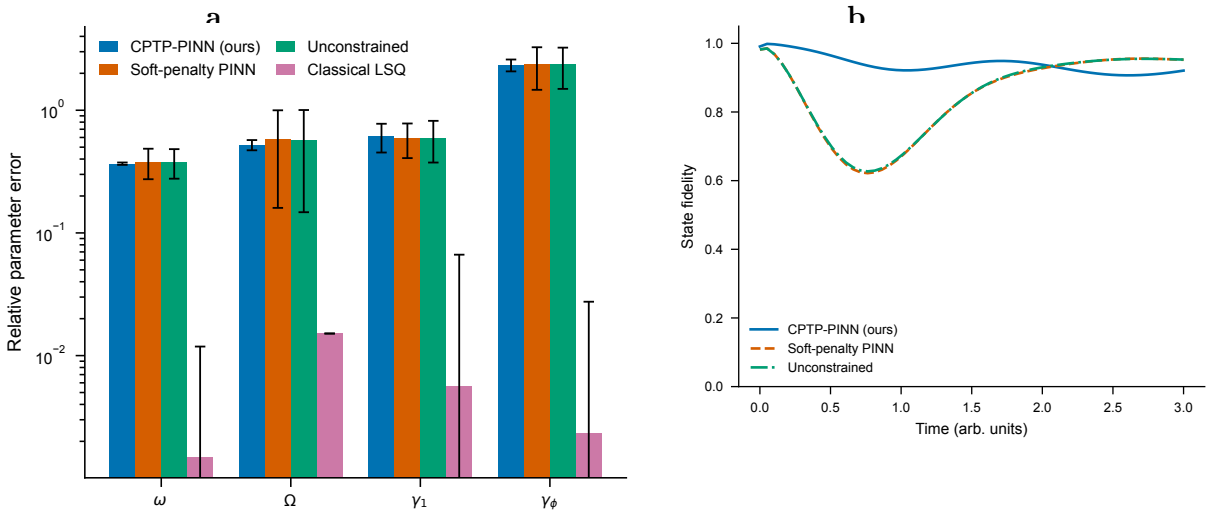

Figure 2: Single-qubit system identification. **a**, Relative parameter-recovery error per method (mean and 95% confidence interval over five seeds, log scale): the CPTP-PINN strongly outperforms the soft-penalty and unconstrained networks and is competitive with a classical least-squares fit that already knows the exact model (Table 1). **b**, State fidelity between the learned and exact density-matrix trajectories (representative seed); the hard-constrained CPTP-PINN stays inside $\mathcal{D}(\mathcal{H})$ by construction. Error bars and bands are 95% confidence intervals over five random seeds.

Table 2: Experiment 2: mean trace distance to the exact trajectory versus retained observation fraction. Mean ± 95% CI over 2 seeds.

Fraction	CPTP-PINN (ours)	Soft-penalty PINN	Unconstrained
1.00	0.2477 ± 0.2332	0.4531 ± 1.4624	0.4539 ± 1.4530
0.50	0.2453 ± 0.2203	0.4542 ± 1.4484	0.4552 ± 1.4359
0.25	0.2536 ± 0.3121	0.4547 ± 1.4426	0.4557 ± 1.4295
0.10	0.2528 ± 0.0628	0.4516 ± 1.4814	0.4522 ± 1.4749

noise. The Bohr frequencies of H drive fast coherent oscillations on top of dissipative decay, and a plain density network suffers from spectral bias; a natural remedy is to encode time with a deterministic Fourier feature basis $t \mapsto [t, \sin(k\omega_0 t), \cos(k\omega_0 t)]_{k=1}^{K}$, $\omega_0 = 2\pi/T$, before the multilayer perceptron. This is the regime where our hard-constrained surrogate is least reliable, and reporting it honestly is one of the points of the multi-seed protocol. Extended Data Table 3 and Figure 3b compare an otherwise-identical density network with $K = 0$ (plain MLP) and $K = 48$ (Fourier): although a Fourier network can fit a favourable run well (single-seed mean fidelity up to 0.95), *across five seeds the reconstruction is unreliable and the Fourier advantage does not survive seed averaging*—mean state fidelity is 0.77 ± 0.35 for the plain MLP and 0.52 ± 0.49 for the Fourier network, with intervals so wide that neither encoding can be claimed to reconstruct the fast dynamics robustly. The Cholesky parameterization still keeps every output a valid density matrix, but physicality alone does not buy accuracy here. We therefore report fast multi-level reconstruction—and the still harder problem of identifying the individual dissipative rates, whose signal is small relative to the coherent dynamics (Sec. 4.8)—as the principal open problem of this work (Sec. 3.1); interaction-picture residuals and derivative denoising are the natural next steps. The two-qubit experiment below shows the complementary positive case: when the learned *generator* (rather than the network trajectory) is the object of interest, the same time encoding suffices to recover a model that generalizes.

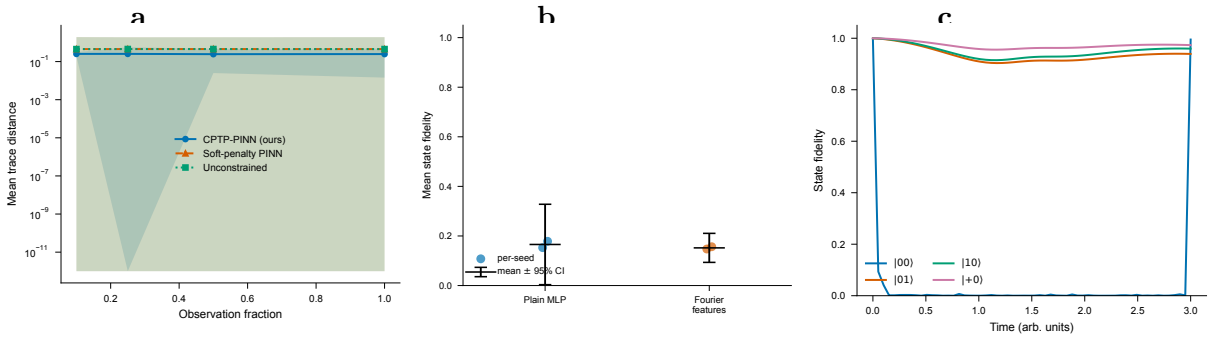

Figure 3: Robustness, limits and generalization across systems. **a**, Sparse-measurement ablation on the single qubit: mean trace distance to the exact trajectory versus retained observation fraction (log scale), for the CPTP-PINN, a soft-penalty PINN and an unconstrained network. **b**, Fast qutrit reconstruction is unreliable across seeds: per-seed mean state fidelity for the plain MLP and the Fourier-feature density network: the single-seed Fourier advantage does not survive seed averaging (the principal open problem). **c**, Two-qubit dissipative gate: state fidelity for the trained $|00\rangle$ trajectory and for three held-out initial states obtained by propagating the learned generator (representative seed). Error bars/bands and points are over five random seeds; see Extended Data Tables 2–4.

2.8 A two-qubit dissipative gate generalizes across initial states

The two-qubit experiment uses a time-dependent exchange coupling $J(t) = J_0 \exp[-(t-1.5)^2/(2\cdot 0.3^2)]$ with $J_0 = 2\pi(0.05)$, local detunings $\Delta_1 = 2\pi(5.0)$, $\Delta_2 = 2\pi(4.8)$, and local amplitude damping and dephasing on each qubit; Fourier time-features handle the fast detuning dynamics. The model is trained on the $|00\rangle$ initial state. The scientifically meaningful test is whether the *learned generator*—rather than the single trained trajectory—predicts held-out initial states, so we propagate the recovered Lindbladian from $|01\rangle$, $|10\rangle$, and $|+0\rangle$ and compare to the exact dynamics. The network reproduces the trained $|00\rangle$ trajectory at fidelity 0.994 ± 0.001, and the recovered generator generalizes to the three held-out states at mean fidelities 0.95, 0.96, and 0.97 (overall average 0.97 ± 0.005; Extended Data Table 4, Figure 3c). In contrast to the direct qutrit reconstruction of Sec. 2.7, this result is stable across seeds (note the narrow confidence intervals), because the test propagates the learned *generator* with an exact solver rather than reading off the neural trajectory. This is the property that matters for device characterization—a transferable open-system model rather than a memorized curve—and it holds because physicality and CPTP structure are enforced by construction while the few generator parameters are shared across all initial states.

2.9 Dense-versus-structured compiler scaling

The compiler experiment compares dense superoperator evaluation against structured matrix-free residual evaluation on random Lindblad systems: dense mode forms the $d^2 \times d^2$ Liouvillian, structured mode evaluates commutator and dissipator terms directly from H and the jump operators. In the CPU benchmark (Extended Data Table 5, Figure 4) the structured evaluation is faster than dense at every tested dimension—a several-fold speedup at small d that grows toward an order of magnitude by $d = 8$—and uses substantially less memory as d grows, while the two residuals agree to floating-point precision. This matches Theorem 6: dense storage scales as d^4, whereas the structured representation stores only H, the m jump operators, and ρ. Timing is hardware dependent; we report the median over repeated runs on a single CPU and emphasize the asymptotic memory scaling as the robust conclusion.

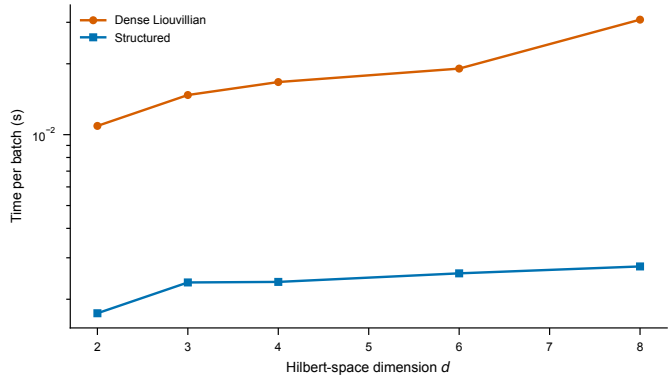

Figure 4: Matrix-free compiler scaling. Dense superoperator residual-evaluation time versus structured matrix-free evaluation (median over 50 repeats, log scale). The structured path is faster at all tested dimensions and avoids materializing the $d^2 \times d^2$ Liouvillian; see Extended Data Table 5.

3 Discussion

What the hard constraint buys. The Cholesky map changes the feasible set. A soft penalty for positivity competes with data fit, residual fit, and optimizer noise; a hard parameterization removes that competition. On the full-data single-qubit problem this is already visible in the *neural* comparison: the CPTP-PINN recovers every parameter far more accurately than the soft-penalty and unconstrained networks and never leaves the physical set, while those baselines do. The only method that beats it there is a classical least-squares fit that already knows the exact model—an honest reference we report rather than omit, and one whose advantage is confined to the dense-data, low-parameter setting. The architectural constraint pays off most clearly in two places the experiments isolate: it guarantees a physical trajectory at every step (zero positivity violations), and it degrades gracefully under sparse measurements (Sec. 2.6), where the soft and unconstrained surrogates leave or fight the positive cone. In larger systems, where the positive cone is a small subset of Hermitian trace-one matrices, this gap is expected to widen.

What the compiler buys. The compiler determines whether the residual can be evaluated without forming an object whose size scales as d^4. In a multi-qubit Hilbert space $d = 2^n$, so dense Liouvillian storage scales as 16^n; structured kernels still face exponential Hilbert-space growth but avoid an unnecessary square of the density-vector dimension. This is the difference between a prototype that only runs on toy models and a path that can exploit locality, batching, sparsity, tensor products, and GPU memory hierarchy. The most direct accelerator path lowers each Hamiltonian and dissipator term to batched matrix multiplications, fuses anticommutator terms, and keeps density matrices resident on GPU; circuit- or pulse-level noise hypotheses can be compiled into Hamiltonian and jump-operator terms via existing stacks [18, 19], and resource-estimation tools may interact with learned models [35, 36].

Optimization geometry and failure modes. The Cholesky map is nonlinear and redundant; near rank-deficient states, small changes in the factor can produce small eigenvalues and ill-conditioning. This is a reason for careful initialization, diagonal floors, learning-rate schedules, and low-rank-plus-isotropic variants, not a reason to abandon the representation. The framework has three distinct diagnostics that should not be conflated (Methods, Sec. 4.8): state physicality (guaranteed), residual size, and held-out prediction error. A model can be physical at every time and still solve the wrong dynamics; only the residual and held-out validation certify the model, and only an identifiability analysis (Theorem 5) certifies the parameters.

3.1 Limitations

1. **Fast multi-level regimes (the principal open problem).** A smooth-in-time density network suffers spectral bias on rapidly oscillating dynamics. Fourier time-features are enough to recover a *generator* that generalizes when it is propagated with an exact solver (the two-qubit gate, Sec. 2.8), but *direct* reconstruction of fast qutrit trajectories is not reliable across seeds (Sec. 2.7): the single-seed Fourier advantage does not survive seed averaging, and the variance is large. Identifying the small dissipative rates there is harder still—the dissipative signal is a small fraction of the coherent dynamics, and the learned network derivative is not yet accurate enough under measurement noise to pin the rates. (A least-squares fit of the linear-in-rates dissipator to the *exact* trajectory recovers the rates to better than 1%, so the rate problem is one of optimization/derivative quality, not identifiability.) Interaction-picture residuals, derivative denoising, more robust initialization, and richer measurement designs are the natural next steps.

2. **No hardware data.** The experiments are synthetic CPU simulations; they test the implementation and theory but do not demonstrate performance on a quantum device.

3. **Markovian assumption.** The framework assumes GKSL dynamics. Non-Markovian systems require extended state spaces, memory kernels, or process tensors.

4. **Identifiability is measurement-dependent.** The theorems cannot overcome uninformative observables: if the sensitivity Gramian is rank deficient, no optimizer can recover all parameters.

5. **Scope of the empirical claims.** The study is a small proof of concept (five seeds per experiment); it is not benchmarked against every modern system-identification method, and it does not claim state of the art.

Conclusion. CPTP-Compiler-PINNs combine physical state parameterization, GKSL-preserving generator learning, and matrix-free residual compilation. The main conceptual point is that physicality should be an architectural invariant rather than an after-the-fact penalty; the mathematics shows this invariant does not destroy universal approximation, that the learned generator remains CPTP, that residuals provide an a posteriori trace-norm certificate, and that structured compilation avoids dense Liouvillian storage. The experiments are deliberately modest but authentic and statistically reported. Future work should extend to larger multi-qubit systems, hardware-calibration datasets, non-Markovian embeddings, and GPU-native kernels, with the most promising engineering direction being a compiler that takes a high-level noise model and emits fused, batched, differentiable kernels for accelerator execution.

4 Methods

4.1 Background: GKSL dynamics and dense Liouvillian

Let $\rho(t) \in \mathcal{D}(\mathcal{H})$. A time-independent Markovian master equation has the form

$$\frac{\mathrm{d}\rho}{\mathrm{d}t} = \mathcal{L}_\Theta[\rho] = -\mathrm{i}[H_\Theta, \rho] + \sum_{k=1}^{m} \gamma_k \Big(L_k \rho L_k^\dagger - \tfrac{1}{2}\{L_k^\dagger L_k, \rho\} \Big), \tag{8}$$

with $H_\Theta = H_\Theta^\dagger$, $\gamma_k \geq 0$; the GKSL theorem characterizes generators of quantum dynamical semigroups in finite dimensions [1–4]. Using the column-major identity $\mathrm{vec}(A\rho B) = (B^T \otimes A)\,\mathrm{vec}(\rho)$, the dense Liouvillian is

$$\mathbb{L}_\Theta = -\mathrm{i}(I \otimes H_\Theta - H_\Theta^T \otimes I) + \sum_k \gamma_k \Big(L_k^* \otimes L_k - \tfrac{1}{2} I \otimes L_k^\dagger L_k - \tfrac{1}{2}(L_k^\dagger L_k)^T \otimes I \Big), \tag{9}$$

which stores d^4 complex scalars, whereas direct evaluation of (8) stores H and the L_k, i.e. $O(md^2)$.

4.2 Generator parameterization (details)

For known jump operators with unknown rates,

$$\gamma_k(\xi_k) = \mathrm{softplus}(\xi_k) + \gamma_{\min}, \qquad \gamma_{\min} \geq 0; \tag{10}$$

for a full Kossakowski matrix in a fixed basis $\{F_a\}_{a=1}^q$, $C_\eta = B_\eta B_\eta^\dagger \succeq 0$ and

$$\mathcal{L}_\Theta[\rho] = -\mathrm{i}[H_\Theta, \rho] + \sum_{a,b=1}^{q} (C_\eta)_{ab}\Big(F_a \rho F_b^\dagger - \tfrac{1}{2}\{F_b^\dagger F_a, \rho\}\Big), \tag{11}$$

with $H_\theta(t) = \sum_\ell \theta_\ell(t) G_\ell$, $G_\ell = G_\ell^\dagger$.

4.3 Observation model and loss (details)

With observables $\{O_j\}_{j=1}^p$, measurements y_{ij} at time t_i, and masks $m_{ij} \in \{0,1\}$,

$$\mathcal{J}_{\mathrm{data}}(\phi) = \frac{1}{\sum_{i,j} m_{ij}} \sum_{i,j} m_{ij}(\mathrm{Tr}[O_j \rho_\phi(t_i)] - y_{ij})^2, \quad \mathcal{J}_0(\phi) = \|\rho_\phi(0) - \rho_0\|_F^2, \tag{12}$$

and $\mathcal{J}_{\mathrm{phys}}$ as in (4). The derivative $\dot{\rho_\phi}$ is obtained by automatic differentiation; the centered finite-difference surrogate $\dot{\rho_\phi}(t) \approx [\rho_\phi(t+h) - \rho_\phi(t-h)]/2h$ is an alternative. The soft-penalty baseline adds $\lambda_{\mathrm{pos}} \sum_i \mathrm{ReLU}(-\lambda_i(\rho))$ to the loss. We use Adam [34]; L-BFGS is a natural second stage [37].

4.4 Open-system IR, dispatch, and algorithms

The compiler receives an IR $\mathsf{IR} = (d, \{G_\ell\}, \{F_a\}, \Theta, \{O_j\}, \mathsf{batch}, \mathsf{mode})$. Dense mode constructs (9); structured mode evaluates (8) directly and is the default when d is large, m is small, or the computation is batched. A practical dispatch rule is

$$\mathsf{mode} = \begin{cases} \mathsf{dense}, & d \leq d_{\mathrm{dense}} \text{ and } N_{\mathrm{batch}} \text{ small}, \\ \mathsf{structured}, & md^2 \ll d^4 \text{ or } N_{\mathrm{batch}} \text{ large}, \\ \mathsf{tensor}, & H, L_k \text{ are sums of tensor-local terms}. \end{cases} \tag{13}$$

Thresholds should be empirical because BLAS libraries, GPU memory, and batch sizes move the crossover.

4.5 Proofs

Proof of Theorem 1. Hermiticity follows from $(AA^\dagger)^\dagger = AA^\dagger$. For every v, $v^\dagger AA^\dagger v = \|A^\dagger v\|_2^2 \geq 0$, so $AA^\dagger \succeq 0$. The denominator is positive because $A \neq 0$ implies $\mathrm{Tr}(AA^\dagger) = \|A\|_F^2 > 0$; normalization gives unit trace. Differentiability follows from the quotient rule on a nonzero denominator, and differentiating $\mathrm{Tr}\,\rho_A = 1$ gives $\mathrm{Tr}\,\dot{\rho}_A = 0$. □

Proof of Theorem 2. Fix $\eta \in (0,1)$ and set $\rho_\eta(t) = (1-\eta)\rho(t) + \eta\sigma \succ 0$, with $\sup_t \|\rho_\eta - \rho\|_F = \eta \sup_t \|\sigma - \rho\|_F$; choose η so this is $< \varepsilon/2$. Since ρ_η is continuous and uniformly positive definite on compact $[0,T]$, its Cholesky factor $C_\eta(t)$ is continuous with positive diagonal, and its real/imaginary lower-triangular entries define a continuous map $[0,T] \to \mathbb{R}^{d^2}$. Universal approximation for feedforward networks [38, 39] gives a finite network approximating this map

Algorithm 1 Residual-kernel dispatch from an open-system IR

Require: IR with dimension d, jump count m, sparsity/tensor metadata, batch size N_b, device.

1: **if** tensor-local metadata available and tensor backend enabled **then**
2: return tensor-local residual kernel.
3: **else if** d^4 storage fits device memory and $d \le d_{\text{dense}}$ **then**
4: materialize dense Liouvillian; return dense matvec kernel.
5: **else if** operators are sparse **then**
6: return sparse structured kernel.
7: **else**
8: return dense-matrix structured kernel (commutator and dissipators term by term).
9: **end if**
10: Attach gradient rule: automatic differentiation, custom adjoint, or finite-difference fallback.

Algorithm 2 CPTP-Compiler-PINN training

Require: Observations $\{(t_i, O_j, y_{ij}, m_{ij})\}$, initial state ρ_0, Hamiltonian basis $\{G_\ell\}$, jump/Kossakowski basis, collocation grid $\{\tau_\ell\}$.
Ensure: Density surrogate $\rho_\phi(t)$ and generator parameters Θ.
1: Initialize ϕ and unconstrained generator parameters ξ; compile the IR to a residual evaluator.
2: **for** step $s = 1, \dots, S$ **do**
3: Form $A_\phi(t)$ by (3) and $\rho_\phi(t)$ by (2); map rates by (10) or $C = BB^\dagger$.
4: Evaluate predictions $\mathrm{Tr}[O_j \rho_\phi(t_i)]$, derivative $\dot{\rho}_\phi(\tau_\ell)$, and $\mathcal{L}_\Theta[\rho_\phi(\tau_\ell)]$.
5: Compute $\mathcal{J} = \lambda_{\text{data}}\mathcal{J}_{\text{data}} + \lambda_{\text{phys}}\mathcal{J}_{\text{phys}} + \lambda_0\mathcal{J}_0$; update (ϕ, ξ) with Adam.
6: **end for**
7: Return (ϕ, Θ) and the residual certificate of Theorem 4.

uniformly; $A \mapsto AA^\dagger/\,\mathrm{Tr}(AA^\dagger)$ is locally Lipschitz on compacts bounded away from zero, so a sufficiently accurate approximation of C_η yields ρ_η within $\varepsilon/2$. The triangle inequality completes the proof. The result is an approximation (not training) guarantee; optimization remains nonconvex. □

Proof of Theorem 3. Softplus is nonnegative, so all rates are nonnegative, and H is Hermitian by construction; hence (8) is in GKSL form and generates a CPTP semigroup [1, 2]. For (11), $C = BB^\dagger \succeq 0$; diagonalizing $C = U\Lambda U^\dagger$ and setting $\widetilde{L}_r = \sum_a U_{ar}\sqrt{\Lambda_r}F_a$ rewrites the dissipator in standard GKSL form. □

Proof of Theorem 4. Let $e = \widehat{\rho} - \rho$. Variation of constants gives $e(t) = \Phi_t[e(0)] + \int_0^t \Phi_{t-s}[r(s)]\,\mathrm{d}s$. Since $\widehat{\rho}$ is Hermitian trace-one and $\mathcal{L}$ trace preserving, $r(s)$ is Hermitian and traceless. A CPTP map contracts trace distance, and by homogeneity $\|\Phi_t[X]\|_1 \le \|X\|_1$ for Hermitian traceless X: writing the Jordan decomposition $X = X_+ - X_-$ with $\mathrm{Tr}\,X_+ = \mathrm{Tr}\,X_- = a$, $\|X\|_1 = 2a$ and $X = a(\rho_+ - \rho_-)$ for density matrices $\rho_\pm$, so $\frac{1}{2}\|\Phi(X)\|_1 = a \cdot \frac{1}{2}\|\Phi(\rho_+) - \Phi(\rho_-)\|_1 \le a \cdot \frac{1}{2}\|\rho_+ - \rho_-\|_1 = \frac{1}{2}\|X\|_1$. The triangle inequality yields (5). □

Proof of Theorem 5. By the fundamental theorem of calculus, $f(\widehat{\Theta}) - f(\Theta_*) = \big(\int_0^1 J(\Theta_* + \alpha(\widehat{\Theta} - \Theta_*))\mathrm{d}\alpha\big)(\widehat{\Theta} - \Theta_*)$. The averaged Jacobian has smallest singular value $\ge s_{\min}/2$ by the perturbation assumption, so $\delta \ge \left\|f(\widehat{\Theta}) - f(\Theta_*)\right\|_2 \ge \frac{s_{\min}}{2}\left\|\widehat{\Theta} - \Theta_*\right\|_2$. □

Proof of Theorem 6. The dense superoperator has $d^2 \times d^2$ entries, i.e. d^4; each jump contributes Kronecker products of $d \times d$ matrices, $O(d^4)$ per jump; a dense matvec costs $O(d^4)$. Structured

evaluation uses $H\rho, \rho H, L_k\rho, (L_k\rho)L_k^\dagger, (L_k^\dagger L_k)\rho, \rho(L_k^\dagger L_k)$, each $O(d^3)$, repeated m times, with storage of one Hamiltonian plus m jump operators. □

Corollary 7 (Low-rank physicality)**.** *For $A \in \mathbb{C}^{d\times r}$ and $\alpha \geq 0$ not both zero, $\rho_\phi^{(r)}(t) = \big(AA^\dagger + \alpha I/d\big)/\big(\operatorname{Tr}[AA^\dagger] + \alpha\big)$ is Hermitian, positive semidefinite, and unit trace. Both $AA^\dagger$ and $\alpha I/d$ are PSD, their sum is nonzero, and trace normalization gives a density matrix. When $\alpha > 0$ the state is full rank; when $\alpha = 0$ its rank is at most r.*

Derivative of the Cholesky map and a Frobenius certificate. With $M = AA^\dagger$ and $Z = \operatorname{Tr} M$, $\rho = M/Z$ and $\dot\rho = \dot M/Z - M\dot Z/Z^2$ with $\dot M = \dot A A^\dagger + A\dot A^\dagger$, $\dot Z = \operatorname{Tr}\dot M$; thus $\dot\rho$ is Hermitian and traceless, and symmetrizing $\dot\rho \leftarrow (\dot\rho + \dot\rho^\dagger)/2$ is a numerical safeguard. If $\|\mathcal{L}[X]\|_F \leq \Lambda \|X\|_F$ and $\left\|\dot{\widehat\rho} - \mathcal{L}[\widehat\rho]\right\|_F \leq \delta$, Grönwall's inequality gives $\|\widehat\rho(t) - \rho(t)\|_F \leq e^{\Lambda t}\|\widehat\rho(0) - \rho(0)\|_F + \frac{e^{\Lambda t}-1}{\Lambda}\delta$; the trace-norm certificate is usually more meaningful for quantum states, but the Frobenius form is easier to estimate during training.

4.6 Sensitivity equations and low-rank variants

Differentiating the master equation, $S_a(t) = \partial\rho_\Theta(t)/\partial\Theta_a$ obeys $\dot S_a = \mathcal{L}_\Theta[S_a] + (\partial_{\Theta_a}\mathcal{L}_\Theta)[\rho_\Theta]$, $S_a(0) = 0$, and the observation Jacobian of Theorem 5 has entries $J_{(i,j),a} = \operatorname{Tr}[O_j S_a(t_i)]$; the same compiler that evaluates residuals evaluates sensitivities, enabling active experiment design (choosing preparations, observables, and times that maximize the smallest singular value of the sensitivity matrix). For larger d, a rank-r variant uses $A_\phi(t) \in \mathbb{C}^{d\times r}$ with the isotropic floor of Corollary 7, connecting to purification-based simulation.

4.7 Experimental implementation details

All experiments are repeated over seeds $\{0, 1, 2, 3, 4\}$ and reported as mean ± 95% Student-t CI; each seed redraws both the network initialization and the measurement noise. The configuration uses 3000 Adam epochs at learning rate 10^{-3}, 100 time points, and noise standard deviation 0.02. State fidelity is the Uhlmann fidelity computed by a Hermitian eigendecomposition (stable for pure/rank-deficient states).

Single-qubit and sparse-measurement experiments (Secs. 2.5–2.6). The density network (`DensityMatrixNet`) is a four-layer MLP of width 96 (width 64, depth 3 for the sparse study) with GELU activations, mapping t to the entries of a lower-triangular complex Cholesky factor; diagonal entries pass through softplus with a small floor. The soft-penalty and unconstrained baselines (`UnconstrainedDensityNet`) use matched architectures and output the matrix entries directly (the soft-penalty variant adds the eigenvalue-negativity penalty with weight $\lambda_{\text{pos}} = 10$). Loss weights are $\lambda_{\text{data}} = 1$, $\lambda_{\text{phys}} = 0.5$, $\lambda_0 = 10$, with a 60-point (50-point for the sparse study) collocation grid over $t \in [0, 3]$ and the residual derivative obtained by automatic differentiation. The classical least-squares baseline fits $(\omega, \Omega, \gamma_1, \gamma_\phi)$ with `scipy.optimize.least_squares` (trust-region reflective, nonnegativity bounds on rates) against the same noisy observables, propagating the exact Lindblad model at each iteration. The sparse study masks a random fraction of the observation entries.

Qutrit and two-qubit experiments (Secs. 2.7–2.8). The scalar time input is augmented with a deterministic Fourier feature basis $[\,t, \sin(k\omega_0 t), \cos(k\omega_0 t)\,]_{k=1}^K$, $\omega_0 = 2\pi/T$, intended to mitigate the spectral bias of a plain MLP on the coherent (Bohr-frequency) oscillations (it does so for the two-qubit generator but, as Sec. 2.7 shows, not robustly for direct qutrit reconstruction). The qutrit observations are the three populations together with the real and imaginary coherence quadratures of the $(0, 1)$ and $(1, 2)$ sectors, at 2% noise; the reconstruction comparison uses $K = 0$ (plain) and $K = 48$ (Fourier). For the two-qubit gate the network ($K = 32$) is trained on $|00\rangle$; the learned generator (known detunings, learned coupling and rates) is read out and

held-out initial states are propagated with the dense Lindblad solver. A least-squares fit of the linear-in-rates dissipator to the *exact* qutrit trajectory recovers the rates to better than 1%, so the open difficulty there is the accuracy of the learned trajectory's time derivative under noise, not identifiability.

Compiler benchmark (Sec. 2.9). Dense mode uses the Kronecker representation (9); structured mode evaluates commutator and dissipator terms directly. We report the median residual-evaluation time over repeated runs (warm-up excluded) for a batch of states, and the analytic storage for the dense versus structured representations. The relative error between the two evaluations,

$$\frac{\|\mathcal{L}_{\text{dense}}[\rho] - \mathcal{L}_{\text{struct}}[\rho]\|_F}{\max(\|\mathcal{L}_{\text{dense}}[\rho]\|_F\,,\,10^{-15})},$$

is at floating-point precision.

4.8 Generator gauge, failure modes, and interpretation

A Lindblad representation is not unique: unitary rotations of the jump-operator list can leave the dissipator invariant, and for a fixed Kossakowski matrix $C = U\Lambda U^\dagger$ one valid jump set is $\widetilde{L}_r = \sum_a U_{ar}\sqrt{\Lambda_r}F_a$, with $U \to UV$ inside a degenerate eigenspace giving the same generator. Recovery of *individual* jump operators is therefore meaningful only within a fixed basis; the identifiable object is otherwise the generator or channel family. This is why the experiments fix the jump operators and learn a small number of rates. Three diagnostics must be kept separate: state physicality (guaranteed by the Cholesky map), residual size, and held-out prediction error. Common failure modes are residual under-resolution (collocation missing fast features; mitigated by adaptive or causal training [20]), rate sloppiness (near-degenerate observables; small singular values of the sensitivity Gramian), boundary ill-conditioning near pure states (mitigated by an isotropic floor), loss imbalance (mitigated by dynamic weighting), and silent dense fallback (the compiler should log memory before materializing a Liouvillian).

4.9 Accelerator-aware implementation roadmap

A production implementation should separate the model layer (Hamiltonian terms, jump operators, rate transforms, observables, constraints) from the kernel layer (dense, structured, sparse, or tensor-local execution). The structured residual repeats products involving the same ρ and operators, so a fused implementation should combine anticommutator terms, reuse $C_k = L_k^\dagger L_k$, and keep intermediates in registers/shared memory for small d; for batches the natural primitive is strided batched GEMM, and for tensor-local terms a sequence of small contractions. Differentiation can use checkpointing, custom vector–Jacobian products for the Lindblad residual, or the adjoint sensitivity equations above. Density matrices are sensitive to Hermiticity and trace errors, so mixed precision needs safeguards: symmetrize after residual evaluation, compute traces in higher precision, and monitor minimum eigenvalues on validation grids.

Extended Data

The following tables give the full numerical values (mean ± 95% confidence interval over five random seeds) that are summarized graphically in the main text: the qutrit reconstruction fidelities of Figure 3b, the two-qubit gate fidelities of Figure 3c, and the compiler benchmark of Figure 4.

Table 3: Experiment 3: qutrit reconstruction mean state fidelity with a plain versus a Fourier-feature density network. Mean ± 95% CI over 2 seeds.

Density network	Mean state fidelity
Plain MLP	0.1657 ± 0.1621
Fourier time-features	0.1520 ± 0.0585

Table 4: Experiment 4: two-qubit dissipative gate. State fidelity for the trained $|00\rangle$ trajectory and for three held-out initial states obtained by propagating the learned generator. Mean ± 95% CI over 2 seeds.

Initial state	Mean fidelity	Final fidelity
$\|00\rangle$ (trained)	0.0377 ± 0.0130	0.9929 ± 0.0257
$\|01\rangle$ (held-out)	0.9334 ± 0.0224	0.9371 ± 0.0248
$\|10\rangle$ (held-out)	0.9473 ± 0.0057	0.9596 ± 0.0003
$\|+0\rangle$ (held-out)	0.9716 ± 0.0027	0.9732 ± 0.0013
Average	0.7225 ± 0.0044	—

Data availability

This study uses no external datasets. All numerical values, tables, and figures are generated by the accompanying scripts from fixed random seeds and are reproduced by running them.

Code availability

The code that implements the method and regenerates every figure, table, and reported number accompanies this manuscript and will be released in a public repository upon publication.

Competing interests

The author declares no competing interests.

Author contributions

M.H. conceived the study, developed the method and the theoretical results, implemented the software and experiments, analysed the results, and wrote the manuscript.

Funding

The author received no specific funding for this work.

References

[1] Göran Lindblad. On the generators of quantum dynamical semigroups. *Communications in Mathematical Physics*, 48(2):119–130, 1976.

[2] Vittorio Gorini, Andrzej Kossakowski, and E. C. George Sudarshan. Completely positive dynamical semigroups of N-level systems. *Journal of Mathematical Physics*, 17(5):821–825, 1976.

Table 5: Experiment 5: compiler runtime and memory scaling. Times are the median residual-evaluation wall time over 15 repeats (warm-up excluded) for a batch of 100 states; memory is the analytic storage for the dense $d^2 \times d^2$ Liouvillian versus the structured operators. CPU, single machine.

d	Dense (s)	Structured (s)	Speedup	Dense (MB)	Structured (MB)
2	0.0109	0.0017	6.2×	0.000	0.00024
3	0.0147	0.0024	6.3×	0.001	0.00069
4	0.0167	0.0024	7.1×	0.004	0.00122
6	0.0191	0.0026	7.4×	0.020	0.00275
8	0.0308	0.0028	11.2×	0.062	0.00488

[3] Heinz-Peter Breuer and Francesco Petruccione. *The Theory of Open Quantum Systems*. Oxford University Press, 2002.

[4] Howard M Wiseman and Gerard J Milburn. *Quantum Measurement and Control*. Cambridge University Press, 2009.

[5] Philip Krantz, Morten Kjaergaard, Fei Yan, Terry P Orlando, Simon Gustavsson, and William D Oliver. A quantum engineer's guide to superconducting qubits. *Applied Physics Reviews*, 6(2):021318, 2019.

[6] Dietrich Leibfried, Rainer Blatt, Christopher Monroe, and David Wineland. Quantum dynamics of single trapped ions. *Reviews of Modern Physics*, 75(1):281, 2003.

[7] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.

[8] Isaac L Chuang and Michael A Nielsen. Prescription for experimental determination of the dynamics of a quantum black box. *Journal of Modern Optics*, 44(11-12):2455–2467, 1997.

[9] J. F. Poyatos, J. I. Cirac, and P. Zoller. Complete characterization of a quantum process: the two-bit quantum gate. *Physical Review Letters*, 78(2):390, 1997.

[10] George C Knee, Eliot Bolduc, Jonathan Leach, and Erik M Gauger. Quantum process tomography via completely positive and trace-preserving projection. *Physical Review A*, 98(6):062336, 2018.

[11] Nicolas Boulant, Timothy F Havel, Marco A Pravia, and David G Cory. Robust method for estimating the Lindblad operators of a dissipative quantum process from measurements of the density operator at multiple time points. *Physical Review A*, 67(4):042322, 2003.

[12] Jianwei Wang, Stefano Paesani, Raffaele Santagati, Sebastian Knauer, Antonio A Gentile, Nathan Wiebe, Francesco Petruccione, Jeremy L O'Brien, John G Rarity, Anthony Laing, and Mark G Thompson. Experimental quantum Hamiltonian learning. *Nature Physics*, 13 (6):551–555, 2017.

[13] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[14] George Em Karniadakis, Ioannis G Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021.

[15] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.

[16] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 2019.

[17] Pauli Virtanen, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17 (3):261–272, 2020.

[18] QuTiP Developers. QuTiP: Quantum toolbox in python for open-system and hybrid quantum-classical simulation. https://qutip.org, 2024.

[19] IBM Quantum. Qiskit: An open-source framework for quantum computing. https://qiskit.org, 2024.

[20] Sifan Wang, Shyam Sankaran, and Paris Perdikaris. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 421:116813, 2022.

[21] Samuel Greydanus, Misko Dzamba, and Jason Yosinski. Hamiltonian neural networks. *Advances in Neural Information Processing Systems*, 32, 2019.

[22] Miles Cranmer, Sam Greydanus, Stephan Hoyer, Peter Battaglia, David Spergel, and Shirley Ho. Lagrangian neural networks. *ICLR 2020 Workshop on Integration of Deep Neural Models and Differential Equations*, 2020.

[23] Ernst Hairer, Christian Lubich, and Gerhard Wanner. Geometric numerical integration. *Springer Series in Computational Mathematics*, 31, 2006.

[24] Giuseppe Carleo and Matthias Troyer. Solving the quantum many-body problem with artificial neural networks. *Science*, 355(6325):602–606, 2017.

[25] Giacomo Torlai, Guglielmo Mazzola, Juan Carrasquilla, Matthias Troyer, Roger Melko, and Giuseppe Carleo. Neural-network quantum state tomography. *Nature Physics*, 14(5): 447–450, 2018.

[26] Nobuyuki Yoshioka and Ryusuke Hamazaki. Variational neural-network ansatz for steady states in open quantum systems. *Physical Review Letters*, 125(23):230601, 2020.

[27] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.

[28] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhatt, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *International Conference on Learning Representations*, 2021.

[29] Sergey Denisov, Tetyana Laptyeva, Wojciech Tarnowski, Dariusz Chruściński, and Karol Życzkowski. From open quantum walks to unitary quantum walks. *New Journal of Physics*, 21(2):023032, 2019.

[30] Michael M Wolf, Jens Eisert, T S Cubitt, and J Ignacio Cirac. Assessing non-Markovian quantum dynamics. *Physical Review Letters*, 101(15):150402, 2008.

[31] Leonardo Banchi, Edward Grant, Andrea Stokes, and Liam Sheridan. Modelling non-Markovian quantum processes with recurrent neural networks. *New Journal of Physics*, 20 (12):123030, 2018.

[32] Ilya A Luchnikov, Sergey V Vintskevich, Henni Ouerdane, and Sergey N Filippov. Machine learning non-Markovian quantum dynamics. *Physical Review Letters*, 124(14):140502, 2020.

[33] Frank Verstraete, Michael M Wolf, and J Ignacio Cirac. Quantum computation and quantum-state engineering driven by dissipation. *Nature Physics*, 5(9):633–636, 2009.

[34] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations*, 2015.

[35] Michael E Beverland et al. Assessing requirements to scale to practical quantum advantage. *arXiv preprint arXiv:2211.07629*, 2022.

[36] Patrick Rall, Chunhao Wang, and Pawel Wocjan. Quantum algorithms for estimating physical quantities using block encodings. *Physical Review A*, 105:032441, 2023.

[37] Dong C Liu and Jorge Nocedal. On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45(1-3):503–528, 1989.

[38] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.

[39] Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.